Group project assignment

See https://github.com/University-of-Mons/calculator-cucumber-2026

Goal:

- Hands-on experience with an iterative, collaborative, test-driven and quality-driven software evolution process on GitHub
- Follow all best practices of open-source software maintenance
- Extend and existing high-quality code base to a full-fledged client-server web application with REST API and Java backend

Practical:

- Strict deadline!
- Collaborative development in groups of students (size of groups to be determined)
- Evaluation based on individual AND group work
  - Measured by activity visible in the group's public GitHub repository
- Ensure, measure and report the evolution of the code and test quality
- After the project's end, oral presentation to reflect on experience in terms of good and bad practices, lessons learned, problems encountered and how they were overcome.

Group project assignment

Instructions:

- See Wiki
  https://github.com/University-of-Mons/calculator-cucumber-2026/wiki
- Add new functionality/features specified as Issues in the repository
  https://github.com/University-of-Mons/calculator-cucumber-2026/issues
- Use test-driven development practices
  - unit testing (JUnit), behaviour-driven testing (Cucumber), regression testing
- Use best practices for collaborative distributed development through git on GitHub
  - commits, tags, releases, issues, pull requests, badges, discussions, workflow automation, project management, …
- Use automation tools for
  - building, continuous integration, code and test quality analysis, security analysis

Group project assignment

Extend basic Java code for integer arithmetic expressions into a full-fledged calculator

Combination of group work + invididual work

- Collaboratively implement, as a group, a web-based GUI with REST API to access calculator features in the Java backend
- Each group (member) implements all “required” features
- Each group implements as many additional features as time and effort permits
- Incremental development
  - Every implemented feature must result in a new (versioned) functional release
- All features need to be integrated into a single fully functional application
- Continuous integration and test-driven development
  - code quality and tests need to be checked and guaranteed after each commit!

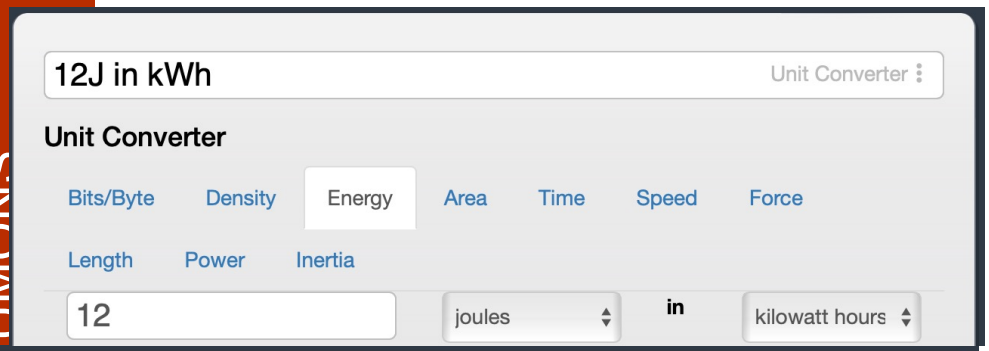
Group project assignment

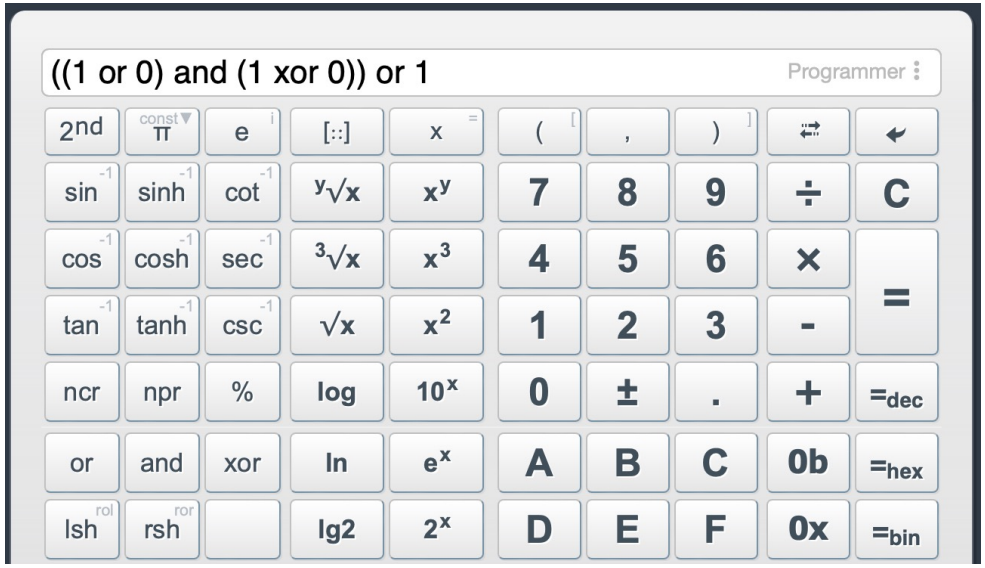
Calculator should support client-server architecture

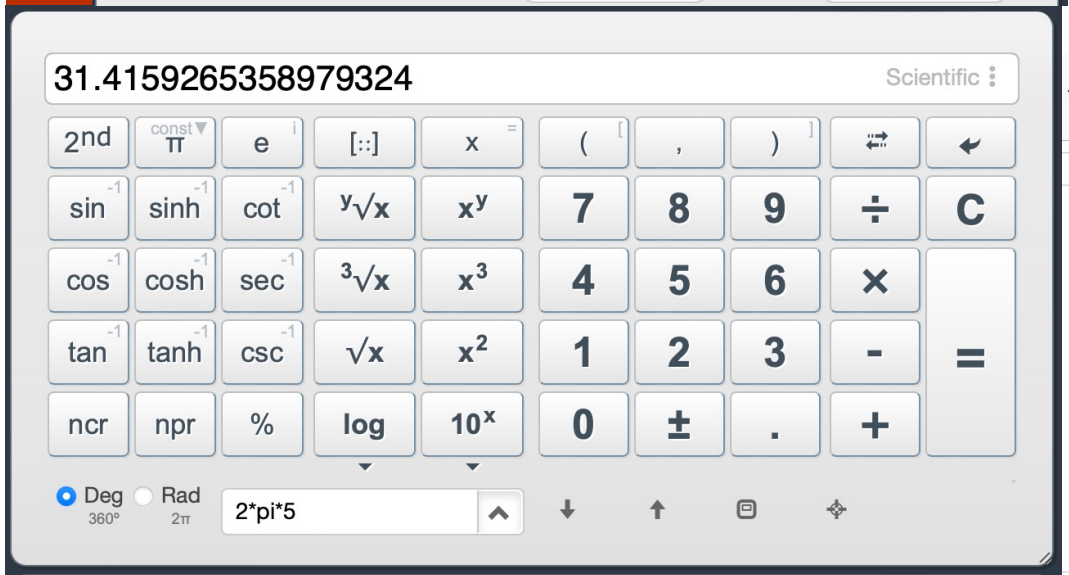
- Backend in Java
  - An expression parser
  - A command-line interface (CLI) based on a read-eval print loop (REPL)
- Frontend
  - web-based GUI running in any web browser
  - Based on some web application framework or library of your choice (in JavaScript, Java, or any other language)
  - Online deployment through AlwaysData (or some alternative solution)
- REST API
  - To allow frontend GUI to query the backend calculator engine

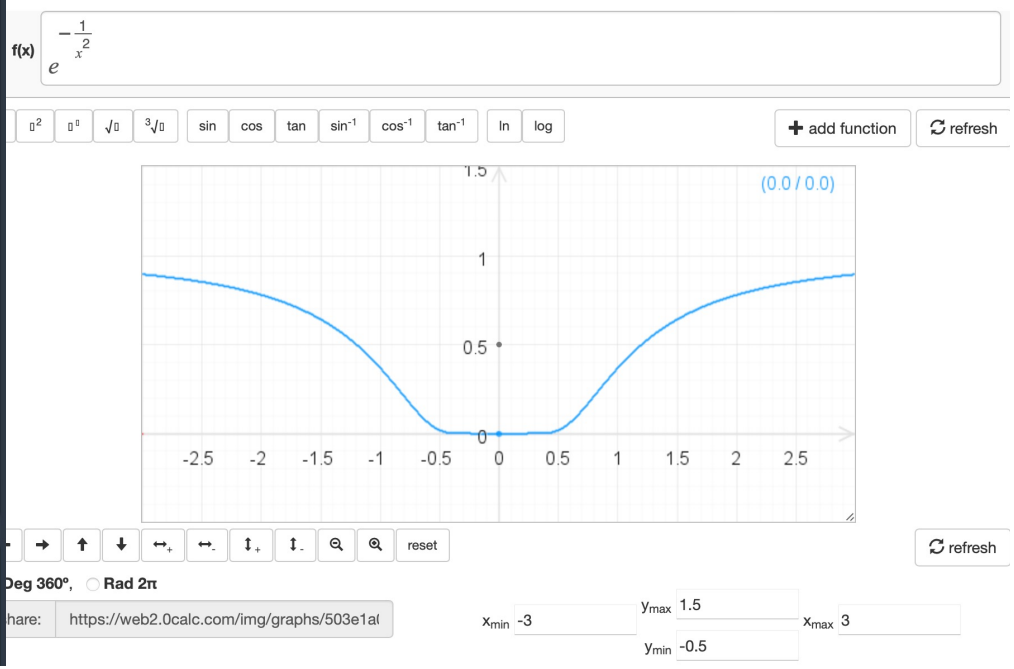
Group project assignment

Some GUI examples as inspiration



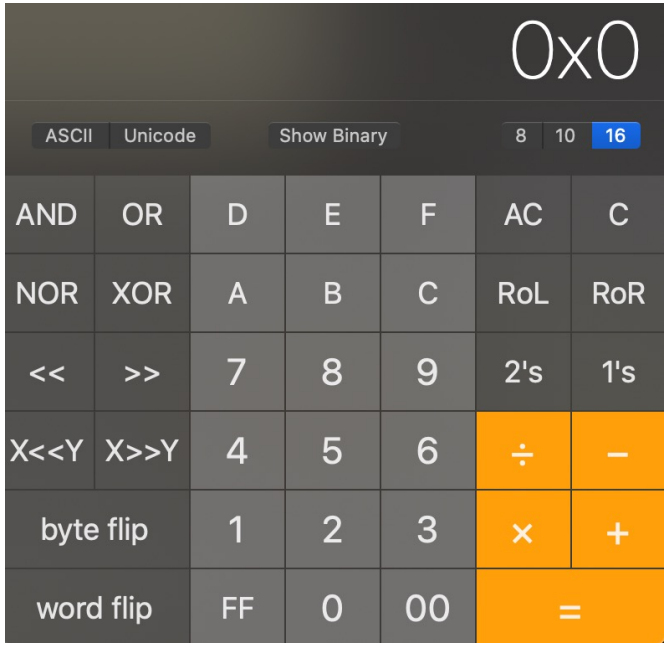


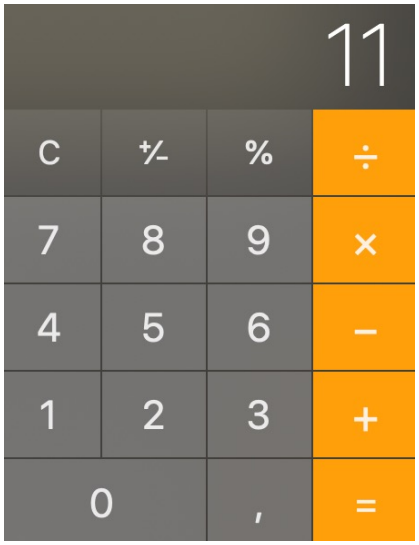


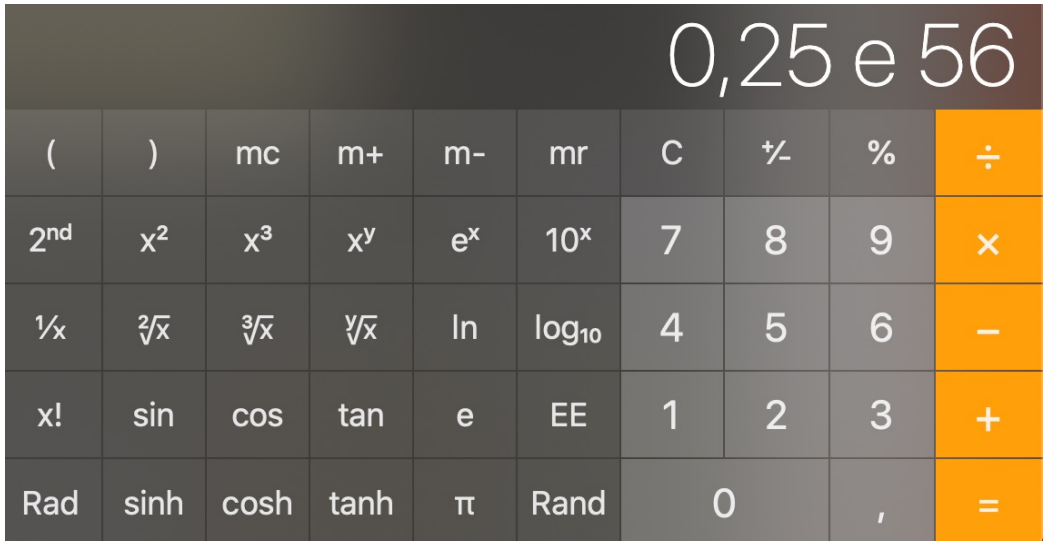


Group project assignment

Some GUI examples as inspiration







Group project assignment

Some command-line interface (CLI) examples with Read-Eval-Print Loop (REPL) as inspiration

```
>>> (1 + 1)
2
>>> (2 * 3 * 4)
24
>>> ((1 + 1) * (5 - 3 - (6 / 2)) * 7)
-14
```

```
> (+ 1 (+ 2 3 (/ 6 4)))
7 1/2
> (* 2+3i 3+4i)
-6+17i
> (* 0+1i 0+1i)
-1
```

```
> (5+4)
[1] 9
> (5+(4*9))
[1] 41
> (5+4)*9
[1] 81
> 5+4*9
[1] 41
> 200-18.7
[1] 181.3
> 5/2
[1] 2.5
> 5 %% 2
[1] 1
> 5 %/% 2
[1] 2
> floor(5/2)
[1] 2
> ceiling(14.7)
[1] 15
> round(pi,2)
[1] 3.14
> log(10)
[1] 2.302585
> log10(10)
[1] 1
> 3^10 / exp(10)
[1] 2.68082
> atanh(tanh(12))
[1] 12
```

Group project assignment

Each group (member) needs to implement features specified as issues in the GitHub repository.

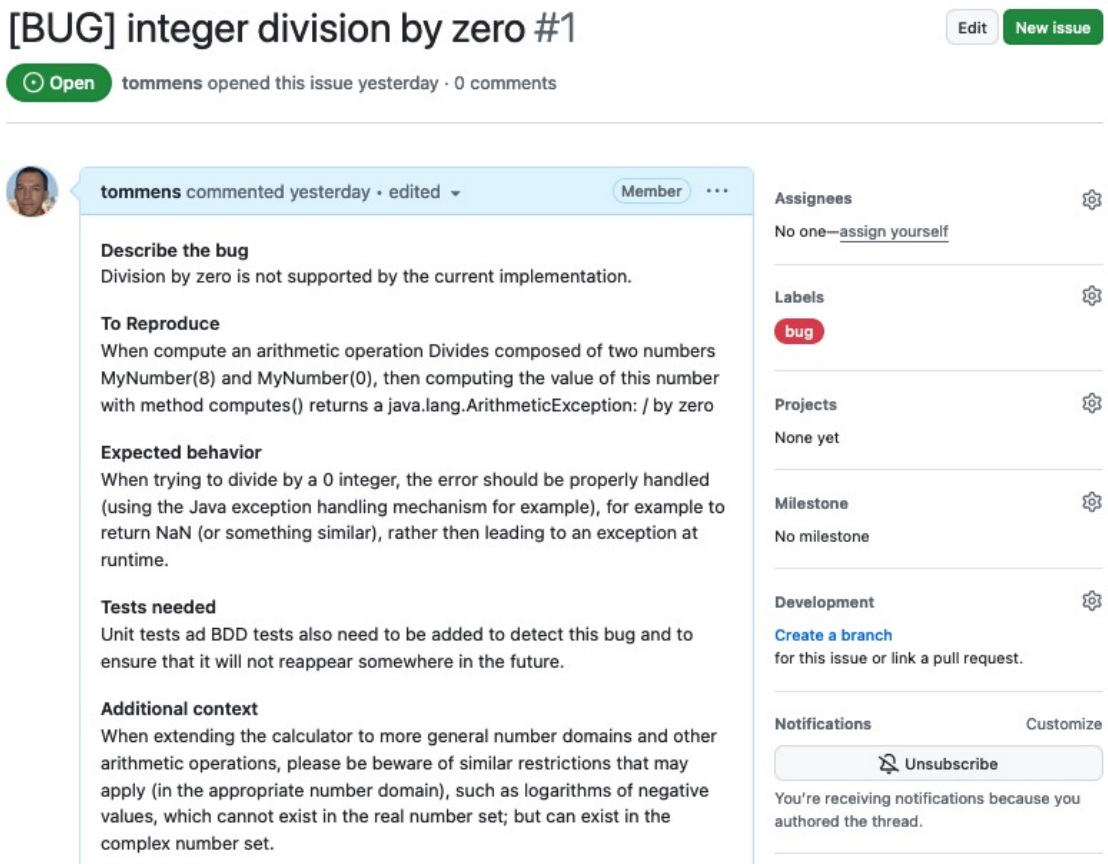
Issues tagged as “required” must be implemented by each group.

Each group MUST implement the CLI and the web-based GUI !

- [FEATURE] Calculator for programmers enhancement
  #16 · tommens opened on Dec 30, 2024
- [FEATURE] More arithmetic operations, constants and random numbers enhancement
  #15 · tommens opened on Dec 30, 2024
- [FEATURE] Supporting different types of numbers enhancement required
  #14 · tommens opened on Dec 29, 2024
- [FEATURE] Mathematical functions and graph drawing enhancement
  #13 · tommens opened on Dec 29, 2024
- [FEATURE] Memory functionality enhancement
  #12 · tommens opened on Dec 29, 2024
- [FEATURE] Conversion of measurement units enhancement
  #11 · tommens opened on Dec 29, 2024
- [FEATURE] Web-based GUI 0 / 3 enhancement required
  #10 · tommens opened on Dec 29, 2024
- [FEATURE] REST API enhancement required
  #9 · tommens opened on Dec 29, 2024
- [FEATURE] Expression parser enhancement required
  #8 · tommens opened on Dec 29, 2024
- [FEATURE] Command-line interface (CLI) and Read-Eval-Print loop (REPL) enhancement required
  #7 · tommens opened on Dec 29, 2024
- [CI] Integrate automated quality analysis automate required
  #5 · tommens opened on Dec 29, 2024
- [CI] Use GitHub Actions to automating testing, quality analysis and code/test coverage when new commits received automate required
  #4 · tommens opened on Dec 29, 2024
- [Refactor] Move pre-/post-/infix output printing functionality to Visitor design pattern refactor required
  #3 · tommens opened on Dec 29, 2024
- [BUG] heterogenous use of in-/post-/prefix notation bug required
  #2 · tommens opened on Dec 29, 2024
- [BUG] integer division by zero bug required
  #1 · tommens opened on Dec 29, 2024

Group project assignment

- Features (or subfeatures) must be assigned to individual group members
- Features can be linked to a *branch or pull request (PR)* addressing the issue



Group project assignment

Implementation should follow a test-first, test-driven development process

Functional testing using Behaviour-Driven Development:
For each feature to be developped:

- Create a .feature file containing executable scenarios of user-observable functionality that will be implemented
- Write the neccesary mapping code to Java
- Run the scenarios after having implemented the functionality

Structural unit testing with
JUnit 5
For each new or modified class or method
– Add unit tests
– Ensure that all unit tests succeed
– Ensure high test coverage

- Optional: Consider GUI testing using
  – e.g., Selenium

Group project assignment

See playlist on YouTube

https://www.youtube.com/playlist?list=PLmHMvhX5wK_YgW76FbnA132nt9o4ChmkD

Software Development

Java

IJ

Build Automation and Continuous Integration

Maven™

GitHub

GitHub Actions

Software Testing

JUnit 5

Quality Analysis

CodeQL

JACOCO
Java Code Coverage

Group project assignment

Process:

Start from the **template repository**
https://github.com/University-of-Mons/calculator-cucumber-2026

Create a new **group repository** from this template to be used for the group project

- Copy the Issues of the template repository to your own.
- Issues tagged as “required” MUST be implemented by each group
- Invite teachers as (read-only) repository collaborators (to follow group progress)
- Teachers will measure and evaluate individual members based on their activity in the group repository
- Group freely decides how to divide work by assigning (sub-)issues to group members to implement, and by keeping track of progress in the project planning on GitHub
- Do not use other repositories, use separate feature branches of the group repository to work on assigned issues

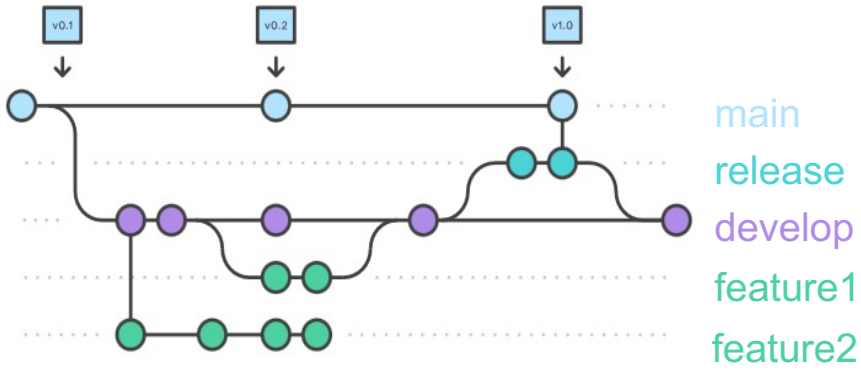
Group project assignment

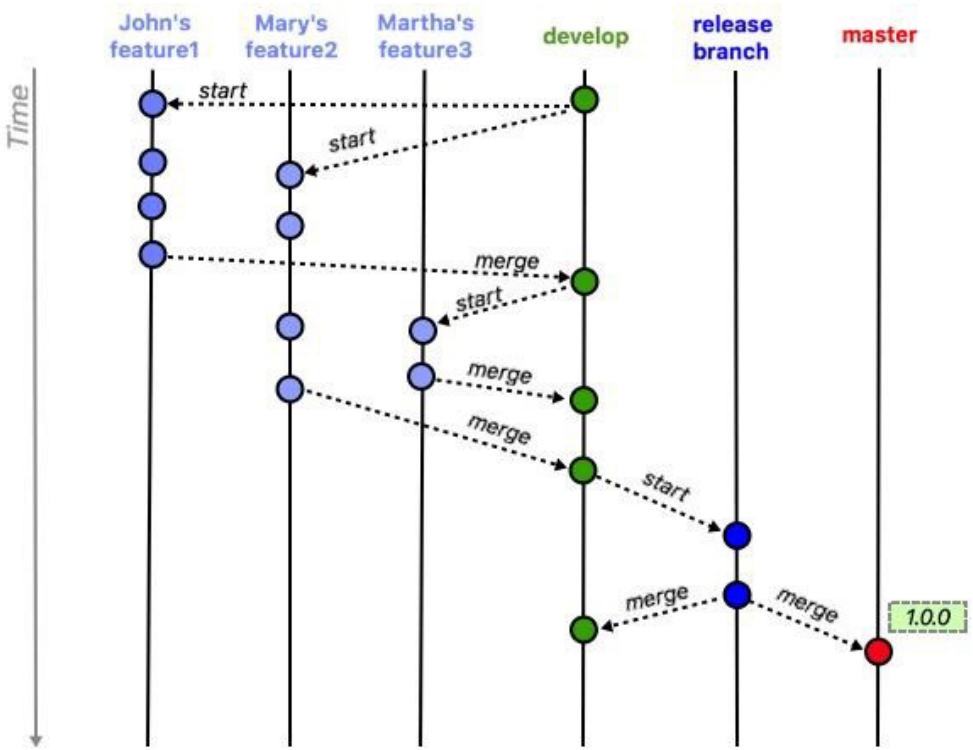
- Deadline for the project: Friday 10 April
- Deliverables:
  - The GitHub repository MUST CONTAIN **multiple releases** in the **main branch** (one for each finished working feature)
    - Do not delete your feature branches after merging, we need them for evaluating individual work.
  - **All** releases MUST HAVE good quality of code and tests, documentation, test coverage, security analysis, ...
  - The fully functional **final release** must
    - implement as much of the features (issues) as possible
    - come with **end-user documentation** explaining how to access and use the application (e.g. in the form of a live demo video, or in the form of a Wiki page with instructions)

- Deadline for the oral presentation: Friday 17 April
- Presentation contents: See Wiki.

Development process
Using branches

- start off from *develop branch*
- use *feature branches* to develop individual features
- integrate ready features into *develop branch*
- prepare new release in a temporary *release branch*
  - no features should be developed during this phase
  - bug fixes, documentation, quality analysis, ...
- publish official release in *main/master branch*

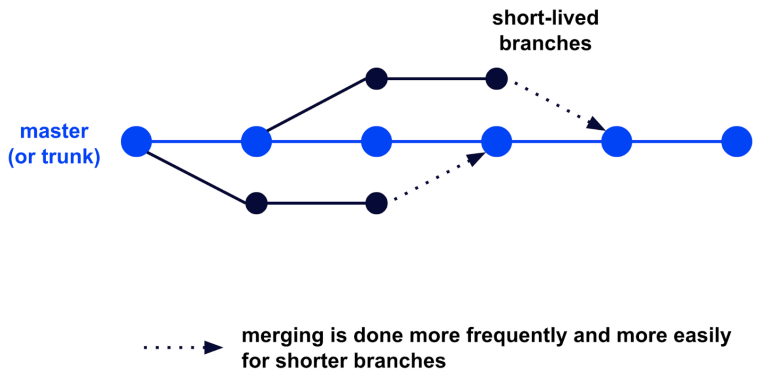


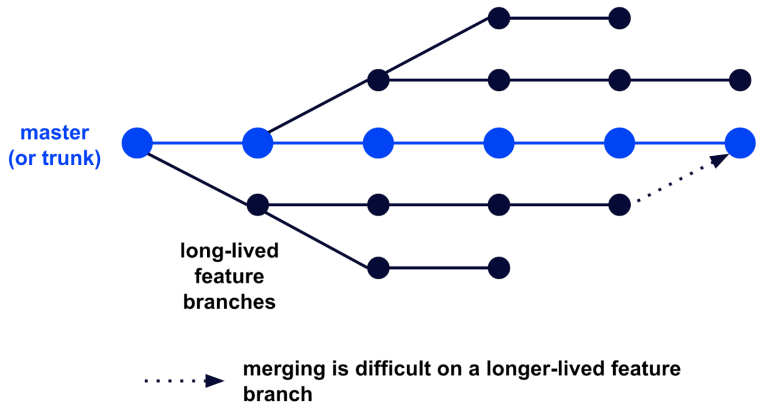


Development process
trunk-based or feature-branched development?

- Trunk-based development
  - A continuous delivery approach to software development
  - Branches are short-lived and merged *as frequently as possible*.
  - Branches are small and often contain only part of a feature to make merging easier because there is less time for divergence between the main trunk and the branches.

- Feature-branched development
  - Individual developers do not merge their branch until a feature is complete, working for weeks or months on a separate copy.
  - This makes merging difficult because the master branch has likely changed due to other merged changes.
  - As a consequence, version control and continuous integration get difficult, involving a lengthy code review process with many different pull requests having to be analyzed to resolve merge conflicts.





Test-First Development

JUnit 5

```java
@BeforeEach
public void setUp() throws Exception {
    List<Expression> params1 =
            new ArrayList<>(Arrays.asList(new MyNumber(3), new MyNumber(4), new MyNumber(5)));
    List<Expression> params2 =
            new ArrayList<>(Arrays.asList(new MyNumber(5), new MyNumber(4)));
    List<Expression> params3 =
            new ArrayList<>(Arrays.asList(new Plus(params1), new Minus(params2), new MyNumber(7)));
    o = new Divides(params3);
    o2 = new Divides(params3);
}

@Test
public void testEquals() {
    assertEquals(o,o2);
}

@Test
public void testCompute() { assertEquals(Integer.valueOf(1), o.compute()); }

@Test
public void testCountDepth() { assertEquals(Integer.valueOf(2), o.countDepth()); }

@Test
public void testCountOps() { assertEquals(Integer.valueOf(3), o.countOps()); }

@Test
public void testCountNbs() { assertEquals(Integer.valueOf(6), o.countNbs()); }
```

Behaviour-Driven Testing with Cucumber

calculator.feature

```
Feature: Integer Arithmetic Expressions
  This feature provides a range of scenarios corresponding to the
  intended external behaviour of arithmetic expressions on integers.

  # This is just a comment.
  # You can start with a Background: that will be run before executing each scenario.

  Background:
    Given I initialise a calculator

  # Each scenario can be seen as a test that can be executed with JUnit,
  # provided that each of the steps (Given, When, And and Then) are
  # implemented in a Java mapping file (CalculatorSteps.Java)

  Scenario: Adding two integer numbers
    Given an integer operation '+'
    When I provide a first number 4
    And I provide a second number 5
    Then the operation evaluates to 9

  Scenario: Subtracting two integer numbers
    Given an integer operation '-'
    When I provide a first number 7
    And I provide a second number 5
    Then the operation evaluates to 2

  Scenario: Multiplying two integer numbers
    Given an integer operation '*'
    When I provide a first number 7
    And I provide a second number 5
    Then the operation evaluates to 35
```

https://cucumber.io/docs/guides/

Tom Mens, UMONS

Group project assignment
Software Testing

Cucumber
Test execution

15 PASSED

100% passed	a few seconds ago	
15 executed	Last Run	
Mac OS X	OpenJDK 64-Bit Server VM 11.0.7+10	

classpath:calculator/calculator.feature

Feature: Integer Arithmetic Expressions

This feature provides a range of scenarios corresponding to the intended external behaviour of arithmetic expressions on integers.

Background:

- **Given** I initialise a calculator

Scenario: Adding two integer numbers

- **Given** an integer operation '+'
- **When** I provide a *first* number 4
- **And** I provide a *second* number 5
- **Then** the operation evaluates to 9

Scenario: Subtracting two integer numbers

- **Given** an integer operation '-'
- **When** I provide a *first* number 7
- **And** I provide a *second* number 5
- **Then** the operation evaluates to 2

Scenario: Multiplying two integer numbers

- **Given** an integer operation '*'
- **When** I provide a *first* number 7
- **And** I provide a *second* number 5
- **Then** the operation evaluates to 35

Test Coverage Analysis

Calculator with Cucumber tests

Calculator with Cucumber tests

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
calculator		89%		91%	5	55	13	108	3	42	0	9
visitor		100%		100%	0	7	0	16	0	5	0	2
Total	43 of 477	90%	2 of 28	92%	5	62	13	124	3	47	0	11

Calculator with Cucumber tests > calculator Source Files

calculator

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Calculator		29%		n/a	2	4	10	14	2	4	0	1
Operation		96%		93%	2	24	3	47	1	15	0	1
MyNumber		96%		87%	1	13	0	17	0	9	0	1
Notation		100%		n/a	0	1	0	1	0	1	0	1
Divides		100%		n/a	0	3	0	7	0	3	0	1
Times		100%		n/a	0	3	0	7	0	3	0	1
Minus		100%		n/a	0	3	0	7	0	3	0	1
Plus		100%		n/a	0	3	0	7	0	3	0	1
IllegalConstruction		100%		n/a	0	1	0	1	0	1	0	1
Total	43 of 407	89%	2 of 24	91%	5	55	13	108	3	42	0	9

Group project assignment
Advanced use of GitHub features

- From git
  - commits, branches, tags
- From GitHub
  - releases
  - badges
  - licenses
  - issues
  - pull requests
  - code reviews
  - discussions
  - projects

- GitHub Automation
  - Quality analysis
  - Dependency analysis
  - Security analysis
  - CoPilot

See dedicated course lecture on this topic for more details